Tolerant Information Retrieval

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Tolerant Information Retrieval

Methods used by search engines to retrieve relevant documents even when a user's query is not perfectly written.

Topics include:

Wildcard Queries

Searching for words using wildcard characters such as *

Example: comput* matches *computer*, *computing*, and *computation*

Spelling Correction

Automatically detecting and correcting spelling mistakes in user queries

Example: retrival → retrieval

Soundex (Phonetic Matching)

Retrieving words that have similar pronunciations but different spellings

Example: *Smith* and *Smyth*

Wildcard Queries

A **wildcard query** allows users to search for information even when they are unsure of the exact spelling or form of a word.

Wildcards help retrieve all possible matching terms from the dictionary.

When the User Is Unsure of the Correct Spelling

Sometimes users know only part of a word or are uncertain about its spelling.

Example: *Sydney* or *Sidney*

Instead of guessing the exact spelling, the user can search using: **S*dney**

This retrieves all matching spellings.

When a Word Has Multiple Accepted Spellings

Some words have different spellings depending on the country or language style.

Examples:

- color / colour
- organize / organise
- center / centre

A wildcard query can retrieve documents containing any of these spelling variations.

When Searching for Different Forms of the Same Word

A user may want to retrieve documents containing different forms of a word that share the same root.

Examples:

- judicial
- judiciary
- judiciaries

Instead of searching each word separately, the user can search using:

- **judicia***

This retrieves all words beginning with **judicia**.

When Searching for Foreign Words or Names

Users may not know the exact spelling of foreign names, places, or institutions.

Example:

- Universität Stuttgart

The user can search using:

- **Universit* Stuttgart**

This retrieves documents containing different spellings or character variations of the word.

Query Processing for Wildcard Queries

- Once a wildcard query has been processed, the system identifies **all dictionary terms that match the wildcard pattern**.
- However, finding the matching terms is only the first step.
- The search engine must still retrieve the documents associated with each matching term before producing the final search results.

Step 1: Find All Matching Terms

- The search engine scans the dictionary and identifies every term that matches the wildcard pattern.
- **Example:**
- Query:
  - se*ate
- Possible matching terms:
  - senate
  - separate
  - sedate

Step 2: Retrieve the Posting Lists

- For each matching term, the search engine retrieves its **posting list** from the inverted index.
- A posting list contains the IDs of all documents in which that term appears.
- For example:

Term	Posting List
senate	D1, D5, D9
separate	D2, D5, D8
sedate	D3, D7

Step 3: Process the Complete Query

- If the search query contains Boolean operators such as **AND**, **OR**, or **NOT**, the search engine must execute these operations using the posting lists of all matching terms.
- **Example Query:**
  - se*ate AND fil*er
- Possible expansions:
  - (senate OR separate OR sedate) AND (filter OR filler OR filtered)
- The search engine retrieves the posting list for every matching term and then performs the required Boolean operations to identify the final set of relevant documents.

Why Is This Computationally Expensive?

- A single wildcard query can match many different words. As the number of matching terms increases:
  - More dictionary entries must be examined.
  - More posting lists must be retrieved.
  - More Boolean operations must be performed.
  - Query processing becomes slower and requires more computational resources.
- For this reason, wildcard queries are generally more expensive than exact keyword searches.

Permuterm Index

- A **Permuterm Index** is a special indexing technique used in Information Retrieval Systems to efficiently process **wildcard queries**, especially when the wildcard (*) appears at the beginning, middle, or end of a search term.
- The main idea is to generate **all possible rotations** of a word and store them in a separate index.
- This allows the search engine to transform complex wildcard queries into simple prefix searches.

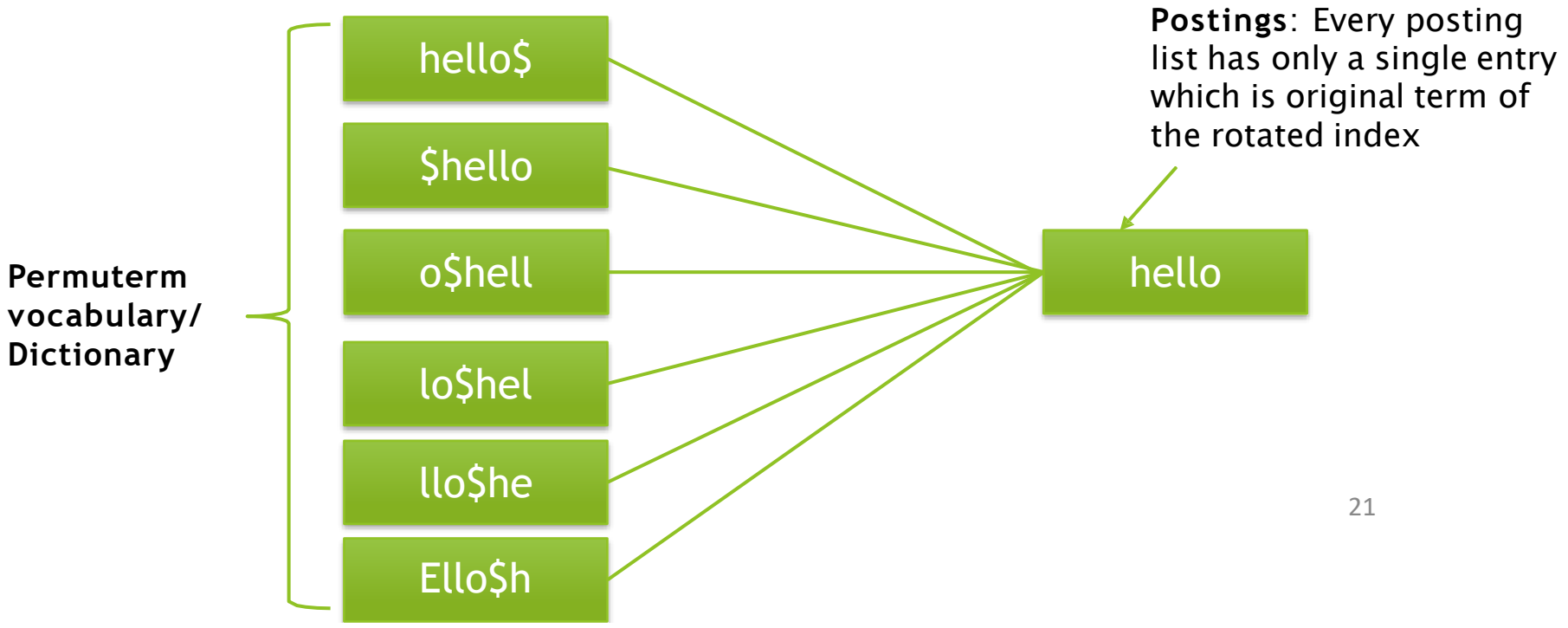
Creating a Permuterm Index

- To create a permuterm index for a term, a special end-of-word symbol ($) is first added to the end of the word.
- The word is then rotated repeatedly until every possible rotation has been generated.
- **Example**
- Original term:
  - hello
- Append the end marker:
  - hello$

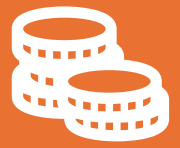
Generate all possible rotations

Rotation	Stored Index Entry
1	hello$
2	ello$h
3	llo$he
4	lo$hel
5	o$hell
6	$hello

Generate all possible rotations



Why Is the $ Symbol Used?



The **$** symbol marks the **end of the original word**.

It allows the search engine to distinguish where the word begins and ends after rotation.

Without this special symbol, different rotations could become ambiguous and difficult to interpret.

Processing Wildcard Queries Using a Permuterm Index

A **Permuterm Index** allows wildcard queries to be processed efficiently by transforming them into **prefix searches**.

The method used depends on the position of the wildcard (*) in the query.

Assume:

- **X** = a string containing one or more characters.
- **Y** = another string containing one or more characters.

Processing Wildcard Queries Using a Permuterm Index

X lookup on **X$**

X* lookup on $**X***

X** lookup on **X$

X lookup on **X***

X*Y lookup on **Y$X***

What Happens After the Lookup?

- Step 1: Find Matching Rotations
- Step 2: Retrieve the Original Terms
  - Each rotated entry points to its original word.
  - For example if the search is **o$hel**

Rotated Entry	Original Word
o$hell	hello
o$help	help
o$helmet	helmet

What Happens After the Lookup?

Step 3: Search the Standard Inverted Index

Step 4: Combine the Results

Limitation of the Permuterm Index

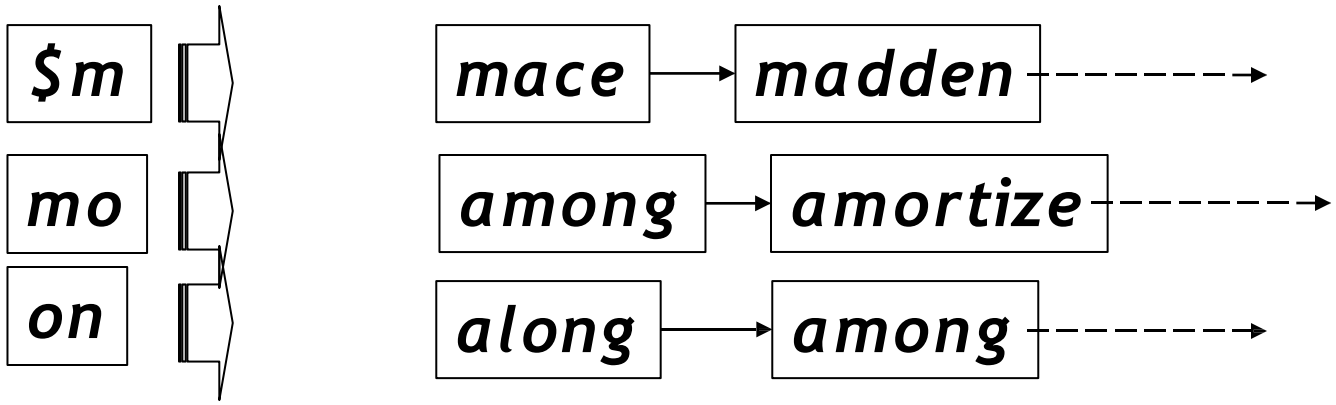
- Although a Permuterm Index provides **fast wildcard query processing**, it has one major disadvantage:
  - **Large Storage Requirement**
- Every word must be stored in **all of its possible rotations**.
- For a word containing **n characters**, the system stores **n + 1 rotated versions**.
- Because every word generates multiple rotated entries, the Permuterm Dictionary becomes much larger than a standard dictionary.
- For **English text**, practical studies have shown that the Permuterm Dictionary is approximately **four times larger** than the original lexicon (dictionary).

k-gram indexes

- Enumerate all k-grams (sequence of k chars) occurring in any term
- Ex- from text "**April is the cruelest month**" we get the 2-grams (bigrams)
- a,ap,pr,ri,il,l,i,is,s,t,th,he,e,$c,cr,ru, ue,el,le,es,st,t$, m,mo,on,nt,h
- Maintain a second inverted index from bigrams to dictionary terms that match each bigram.

Bigram index example

- The *k*-gram index finds *terms* based on a query consisting of *k*-grams (here *k*=2).



Processing wild-cards

Query ***mon**** can now be run as ***$m*** *AND* ***mo*** *AND* ***on***

Gets terms that match AND version of our wildcard query

Candidate Word	Matches mon*?
month	✓ Yes
monitor	✓ Yes
money	✓ Yes
moon	✗ No

False Positives

- Although the candidate words contain all required bigrams, **not all of them satisfy the original wildcard query**.
- Consider the example

k-Gram Index vs. Permuterm Index

Feature	k-Gram Index	Permuterm Index
Storage Requirement	Lower	Higher
Supports Wildcard Queries	Yes	Yes
Needs Post-Filtering	Yes (to remove false positives)	Usually No
Wildcard Processing Speed	Fast	Very Fast
Dictionary Size	Smaller	Much Larger (stores every rotation)

Spell Correction

Spell Correction is a technique used in Information Retrieval Systems to detect and correct spelling mistakes.

It helps improve both the quality of the indexed data and the accuracy of search results.

Principal Uses of Spell Correction

- **Correcting Documents During Indexing**
  - Before documents are added to the search index, spelling mistakes can be detected and corrected.
- **Benefits**
  - Improves the quality of the indexed documents.
  - Prevents incorrect words from entering the dictionary.
  - Increases retrieval accuracy.

Principal Uses of Spell Correction

- **Correcting User Queries**
  - Users frequently make spelling mistakes while searching.
  - The search engine automatically suggests or replaces the misspelled word with the most likely correct word.

Main Methods of Spell Correction

- **Method 1: Isolated Word Spell Correction**
  - In this approach, **each word is checked independently**, without considering the surrounding words.
  - The system compares the misspelled word with words in the dictionary and selects the one with the **smallest edit distance** (the fewest character changes required to transform one word into another).
- **Method 2: Context-Sensitive Spell Correction**
  - Context-sensitive spell correction considers **the surrounding words** in the sentence instead of checking each word individually.
  - The system analyzes the sentence and determines which word best fits the context.

Comparison of the Two Methods

Feature	Isolated Word Correction	Context-Sensitive Correction
Checks each word separately	✓	✗
Uses surrounding words	✗	✓
Detects non-word errors (e.g., retrival)	✓	✓
Detects real-word errors (e.g., form instead of from)	✗	✓
Computational complexity	Lower	Higher
Accuracy	Good	Better

Document correction

Especially needed for OCR’ed documents

- Correction algorithms are tuned for this: r n/m
- Can use domain-specific knowledge
- E.g., OCR can confuse O and D more often than it would confuse O and I (adjacent on the QWERTY keyboard, so more likely interchanged in typing).

But also: web pages and even printed material have typos

Goal: the dictionary contains fewer misspellings

But often we don’t change the documents and instead fix

the query-document mapping

Query Misspellings

Query misspellings occur when users enter search terms with spelling mistakes. If these errors are not handled properly, the search engine may fail to retrieve the relevant documents.

The primary goal of query spelling correction is to help users obtain the information they intended to search for, even when their query contains errors.

Isolated Word Correction

Isolated Word Correction is a spell correction technique in which **each word is checked independently**, without considering the surrounding words or sentence context.

The objective is to find the **correct dictionary word** that is most similar to the misspelled word.

How Does Isolated Word Correction Work?

Step 1: Maintain a Dictionary of Correct Words

- The system stores a collection of correctly spelled words, known as the **dictionary** or **lexicon**.

Examples include:

- Webster's Dictionary
- Oxford English Dictionary (OED)
- A domain-specific dictionary (e.g., medical, legal, or technical terms)

When a user enters a word, the system compares it with the words in this dictionary.

How Does Isolated Word Correction Work?

- **Step 2: Detect a Misspelled Word**
  - Suppose a user enters:
    - retrival
  - Since **retrival** does not exist in the dictionary, the system identifies it as a misspelled word.

How Does Isolated Word Correction Work?

- **Step 3: Compare with Dictionary Words**
  - The system compares the misspelled word with every possible dictionary word and calculates a **similarity (or distance) measure**.
  - The word with the **smallest distance** is considered the most likely correction.
- Example

Misspelled Word	Candidate Word	Distance
retrival	retrieval	1
retrival	retriever	3
retrival	retrievals	4

Measuring Similarity Between Words

To determine which dictionary word is closest to the misspelled word, the system uses one of several similarity measures.

The three most common techniques are:

- Edit Distance (Levenshtein Distance)
- Weighted Edit Distance
- n-Gram Overlap

Edit distance (Levenshtein distance)

- Edit distance between two strings s_1 and s_2 is the minimum number of basic operations that transform s_1 into s_2.
- Levenshtein distance: Acceptable operations are **insert**, **delete** and **replace**

horse → rorse **(REPLACE 'h' with 'r')**

rorse → rose **(DELETE 'r')**

rose → ros **(DELETE 'e')**

Operations

1. REPLACE
2. INSERT
3. DELETE

Edit distance = 3

Ex: DELETE -> REPLACE

		“”	R	E	P	L	A	C	E
“”	0	1	2	3	4	5	6	7
D	1	0+1=1	1+1=2	2+1=3	3+1=4	4+1=5	6	7
E	2	1+1=2	1	2	3	4	5	6
L	3							
E	4							
T	5							
E	6							

If c_1 != c_2

Replace/copy	delete
insert	Minimum cost +1

If c_1 = c_2

Replace/copy	delete
insert	

Ex: DELETE -> REPLACE

	“”	R	E	P	L	A	C	E
“”	0	1	2	3	4	5	6	7
D	1	1	2	3	4	5	6	7
E	2	2	1	2	3	4	5	6
L	3	3	2	2	2	3	4	5
E	4	4	3	3	3	3	4	4
T	5	5	4	4	4	4	4	5
E	6	6	5	5	5	5	5	4

Edit distance = 4

Ex: DELETE -> REPLACE

	“”	R	E	P	L	A	C	E
“”	0	1	2	3	4	5	6	7
D	1	1	2	3	4	5	6	7
E	2	2	1	2	3	4	5	6
L	3	3	2	2	2	3	4	5
E	4	4	3	3	3	3	4	4
T	5	5	4	4	4	4	4	5
E	6	6	5	5	5	5	5	4

Cost	Operation	Input	Output
1	delete	l	*
0	copied	e	e
1	replace	d	r

What Is Weighted Edit Distance?

Weighted Edit Distance is an improved version of the standard **Levenshtein Edit Distance**.

Instead of assigning the same cost to every editing operation, it assigns **different costs (weights)** depending on the characters involved.

The idea is that some mistakes are **more likely to occur** than others.

Therefore, those mistakes should have a **lower cost**.

Why Do We Need Weighted Edit Distance?

The standard Edit Distance treats every insertion, deletion, or substitution equally.

For example:

- Replacing **m** with **n** has a cost of **1**.
- Replacing **m** with **q** also has a cost of **1**.

However, in reality, these two mistakes are **not equally likely**.

Weighted Edit Distance considers how errors actually occur.

Common Sources of Errors

OCR (Optical Character Recognition) Errors

Keyboard Typing Errors

OCR (Optical Character Recognition) Errors

OCR systems often confuse characters that look visually similar.

Examples:

- **m ↔ rn**
- **O ↔ D**
- **0 ↔ O**
- **l ↔ I**

These substitutions receive lower costs because they occur frequently.

Keyboard Typing Errors

People often press nearby keys accidentally.

Example:

- m → n

is much more common than

- m → q

Therefore,

- Cost(m → n) = Low
- Cost(m → q) = High

Probability-Based Approach

Weighted Edit Distance can also be represented using a **probability model**.

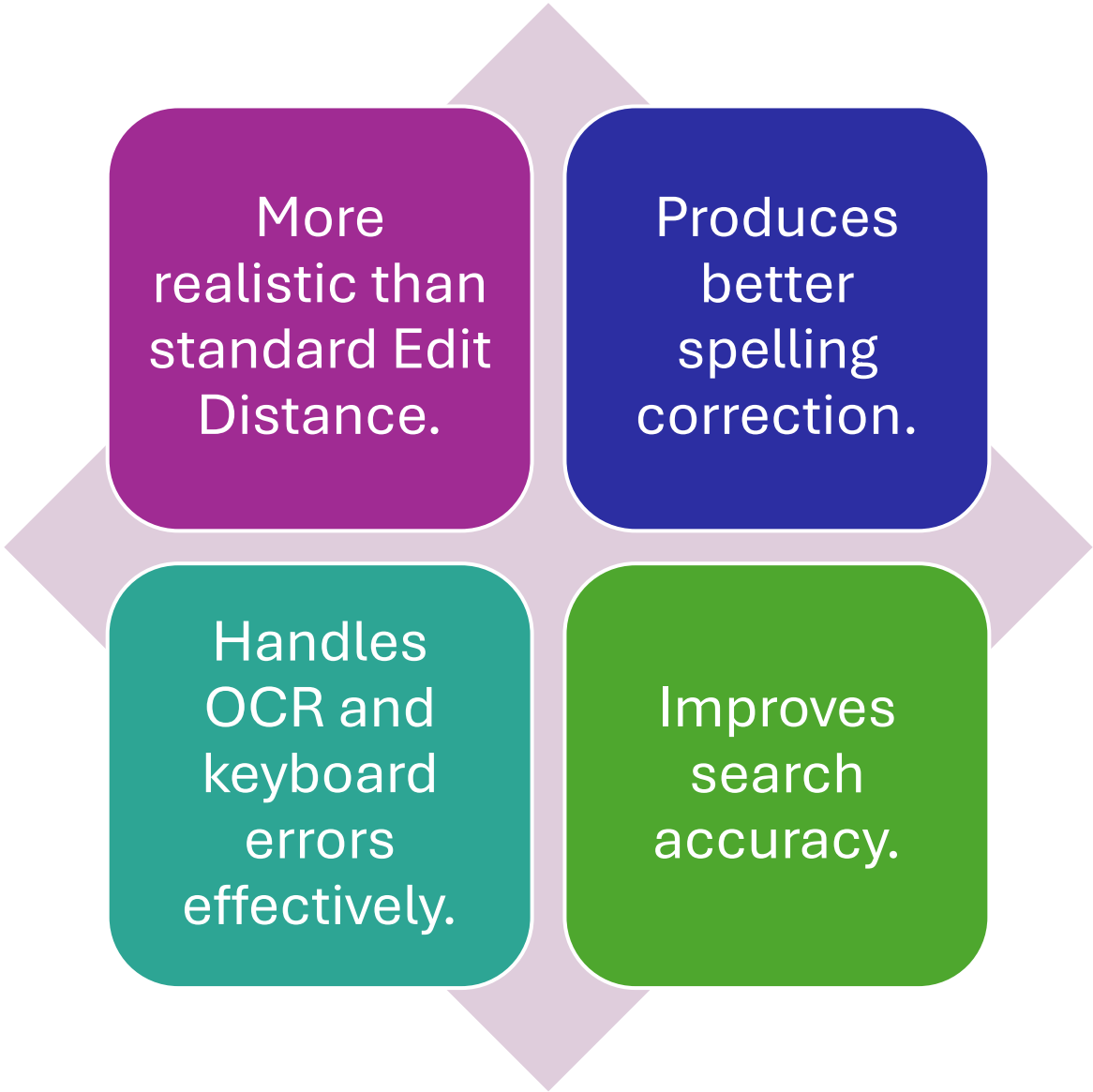
Frequently occurring mistakes receive **smaller weights**, while rare mistakes receive **larger weights**.

To implement this, the algorithm requires a **weight matrix** that stores the cost of transforming one character into another.

Example

FROM	TO	COST
m	n	0.2
m	q	1.5
O	D	0.3
O	X	1.8

Advantages


More realistic than standard Edit Distance.
Produces better spelling correction.
Handles OCR and keyboard errors effectively.
Improves search accuracy.

Edit Distance to All Dictionary Terms?

- Suppose a user enters the misspelled query: retrival
- To correct it, one approach is to calculate the **Edit Distance** between the query and **every word in the dictionary**.
- Consider the given example
- The closest word would then be selected.

DICTIONARY WORD	EDIT DISTANCE
retrieval	1
retriever	3
retrieve	2
review	6
return	5

Why Is This Expensive?

Large search engines contain **millions of dictionary terms**.

Calculating the Edit Distance between the query and every term would require:

- Millions of comparisons.
- High computation time.
- Large memory usage.

As a result, this approach is **too slow** for real-time search systems.

Reducing the Number of Candidate Words

Instead of comparing the query with every dictionary term, the search engine first identifies a **small set of likely candidate words**.

Only those candidates are compared using Edit Distance.

One Effective Solution: n-Gram Overlap

This introduces **n-Gram Overlap** as a method for selecting candidate words.

The idea is simple:

- Break the query into n-grams.
- Find dictionary words that share many of those n-grams.
- Compute Edit Distance only for those words.
- This greatly reduces computation time.

n-Gram Overlap

n-Gram Overlap measures the similarity between two words by counting how many **n-grams** they have in common.

Words that share many n-grams are considered more similar.

Step 1: Generate n-Grams

- The system extracts all n-grams from:
  - The user's query.
  - Every dictionary word.

Step 2: Use the n-Gram Index

The **n-Gram Index** retrieves all dictionary terms containing one or more of the query's n-grams.

Step 3: Count Matching n-Grams

- For every candidate word, count how many n-grams match.
- Example:
  - Query: machine
- Bigrams:
  - ma
  - ac
  - ch
  - hi
  - in
  - ne

Step 3: Count Matching n-Grams

- Candidate word: machien
- Common bigrams:
  - ma
  - ac
  - ch
  - hi
- More shared n-grams indicate greater similarity.

Step 4: Apply a Threshold

- Only words with enough matching n-grams are kept.
- Example:
  - At least 5 matching bigrams.
  - Or at least 80% overlap.
- The remaining candidates are checked using Edit Distance.

Example with Trigrams

Suppose the dictionary contains:

november

The user types:

december

Generate Trigrams

Word 1: november

Trigrams:

- nov
- ove
- vem
- emb
- mbe
- ber

Generate Trigrams

Word 2: december

Trigrams:

- dec
- ece
- cem
- emb
- mbe
- ber

Compare the Trigrams

Common trigrams:

- emb
- mbe
- ber

Total common trigrams = **3**

Each word contains **6 trigrams**.

Why Is This Useful?

Even though the words are different, they share **three consecutive character sequences**, indicating they are related.

Instead of comparing every dictionary word, the search engine can quickly identify **november** as a strong candidate.

Need for Normalization

Simply counting matching trigrams is sometimes misleading.

Words of different lengths naturally contain different numbers of trigrams.

Therefore, we need a **normalized similarity measure**.

The next slide introduces the **Jaccard Coefficient** for this purpose.

Jaccard Coefficient

- If:
  - **X** = Set of n-grams from Word 1
  - **Y** = Set of n-grams from Word 2

$$Jaccard\ Coefficient = \frac{|\ X \cup Y\ |}{|\ X \cap Y\ |}$$

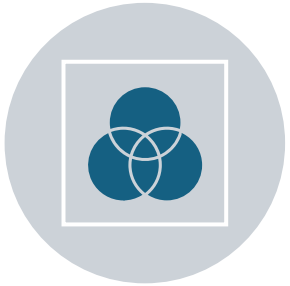
- where:
  - X∩Y = Common n-grams (intersection).
  - X∪Y = Total unique n-grams (union).

Properties

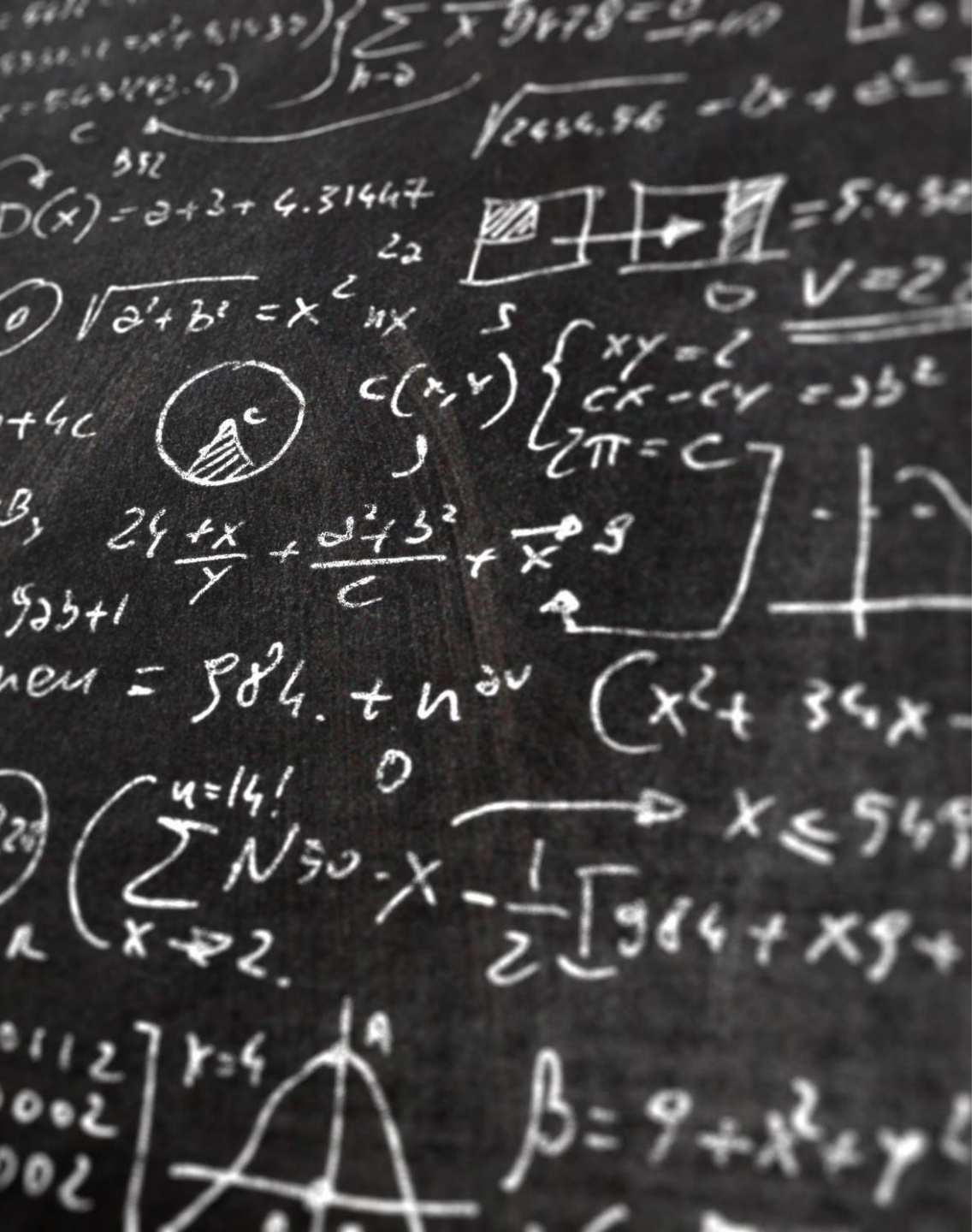
Value is always between **0 and 1**.

1 means the sets are identical.

0 means the sets share nothing.



The two sets do not need to be the same size.



Jaccard Coefficient with n-grams

- The Jaccard Coefficient measures how large this overlap is compared to all unique trigrams from both words.
- A higher value indicates greater similarity.
- Choosing a **threshold** to decide whether two words are considered similar.
- Example:
  - If Jaccard Coefficient > 0.8, then the words are treated as a match.
- The threshold depends on the application's accuracy requirements.

Another Option for n-Gram Matching

Instead of comparing every dictionary word using Edit Distance, we can first identify **candidate words** by counting how many **bigrams (2-grams)** they share with the query.

This approach significantly reduces the number of words that need detailed comparison.

Example

Suppose the user searches for:

- lord

The bigrams are:

- **lo**
- **or**
- **rd**

The search engine now searches the **Bigram Index** to find dictionary words containing these bigrams.

Candidate Words

Word	Matching Bigrams
border	or, rd
morbid	or
card	rd
alone	lo
sloth	lo
ardent	rd

Word	Number of Matching Bigrams	Candidate?
border	2	✓ Yes
morbid	1	✗ No
alone	1	✗ No
card	1	✗ No
sloth	1	✗ No

Selecting Candidate Words

- Instead of requiring all three bigrams to match, the system may select words that match **at least two bigrams**.
- Consider the example.
- Only strong candidates proceed to the next stage.

Ranking Candidates

Candidate words can then be ranked using similarity measures such as:

- Jaccard Coefficient
- Edit Distance
- Weighted Edit Distance

The word with the highest similarity score is selected as the best correction.

Context-Sensitive Spell Correction

Some spelling mistakes cannot be detected by checking words individually because the incorrect word is **already a valid dictionary word**.

These are called **real-word errors**.

Example

Consider the sentence:

I flew from Heathrow to Narita.

Suppose the user searches for:

flew form Heathrow

The word **form** is correctly spelled.

However, it is incorrect in this sentence.

Problem with Isolated Word Correction

An isolated spell checker sees:

- flew ✓
- form ✓
- Heathrow ✓

Since every word exists in the dictionary, it detects **no spelling mistake**.

As a result, the search engine may fail to retrieve the intended documents.

Desired Response

A context-sensitive spell checker recognizes that **form** does not fit naturally after **flew**.

It suggests:

- **Did you mean: flew from Heathrow?**

The corrected query retrieves the intended documents.

How Context-Sensitive Correction Works

Step 1: Find Similar Words

For every word in the query, the search engine first finds similar dictionary words using **Weighted Edit Distance**.

Example:

- Original word:
  - form
- Possible corrections:
  - from
  - farm
  - foam
  - forum

How Context-Sensitive Correction Works

Step 2: Generate Alternative Phrases

Instead of changing every word simultaneously, the system changes **one word at a time**.

Original query:

- flew form Heathrow

Possible alternatives:

- flew **from** Heathrow ✓
- fled form Heathrow
- flea form Heathrow

How Context-Sensitive Correction Works

Step 3: Search Each Alternative

Each alternative phrase is searched in the document collection.

The search engine counts how many documents match each phrase.

This is called **hit-based spelling correction**.

Candidate Phrase	Number of Matching Documents
flew from Heathrow	2,450
fled form Heathrow	8
flea form Heathrow	0

How Context-Sensitive Correction Works

- **Step 4: Select the Best Phrase**
- The phrase that produces the **largest number of matching documents (hits)** is considered the most likely correction.
- Consider the example.
- The search engine selects:
  - **flew from Heathrow**

Exercise

Suppose we have the query:

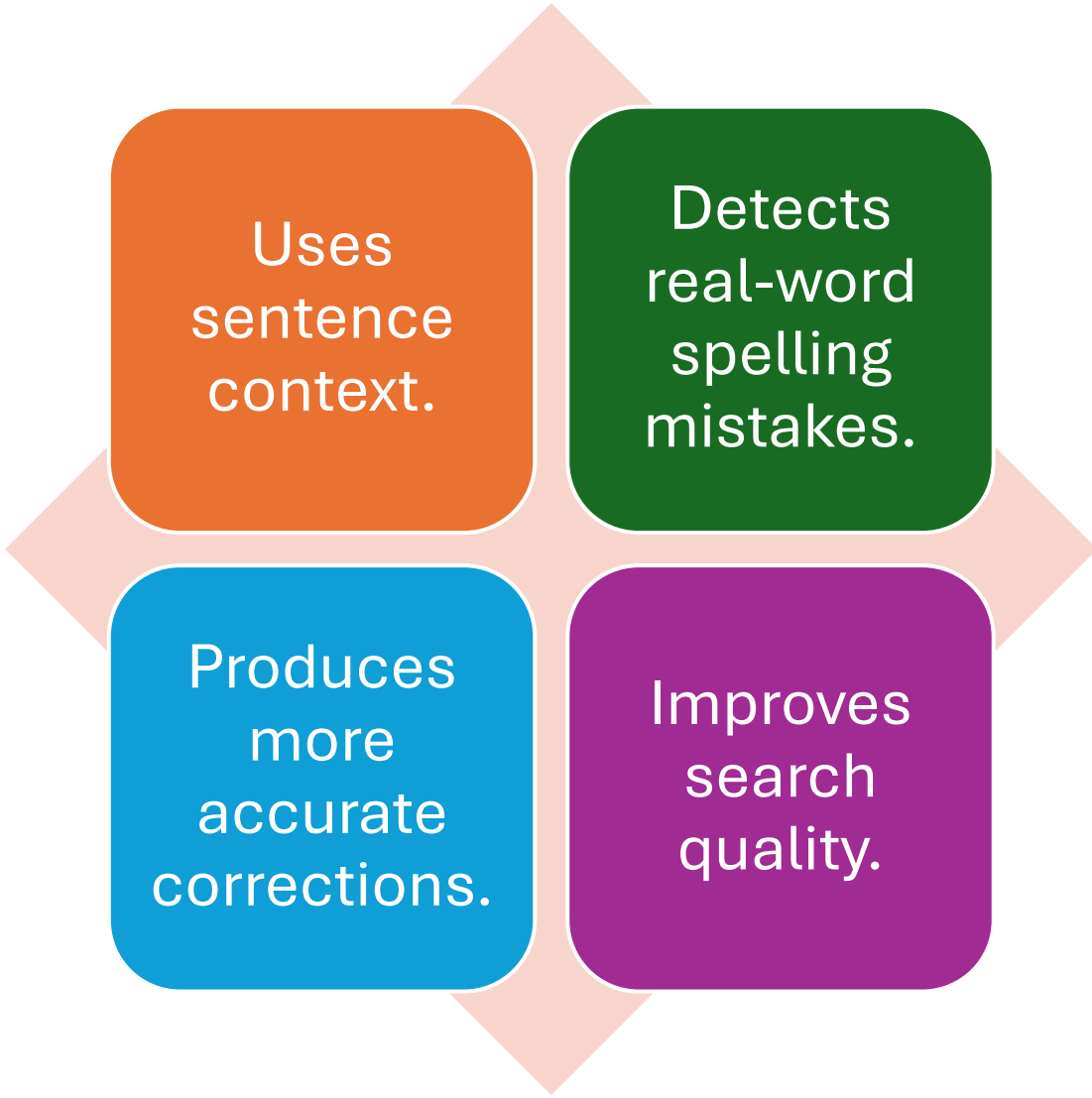
- flew form Heathrow

The spell checker generates the following correction candidates:

- **flew** → 7 possible alternatives
- **form** → 19 possible alternatives
- **Heathrow** → 3 possible alternatives

How many corrected phrases will be generated using the approach discussed earlier?

Advantages


Uses sentence context.
Detects real-word spelling mistakes.
Produces more accurate corrections.
Improves search quality.

Another Approach – Biword Indexing

A **Biword Index** stores **pairs of consecutive words** instead of indexing individual words.

Each entry in the index represents two adjacent words that appear together in a document.

Example

Sentence:

- Machine Learning Models

Biwords:

- Machine Learning
- Learning Models

These pairs are stored in the Biword Index.

Why Is It Useful?

Biword Indexing helps the search engine process **phrase queries** more efficiently.

Instead of searching each word separately, the system directly searches for the stored word pair.

Application in Spell Correction

Suppose the user enters:

- flew form Heathrow

Possible correction:

- flew from Heathrow

The search engine checks whether the biwords:

- flew from
- from Heathrow

exist frequently in the index.

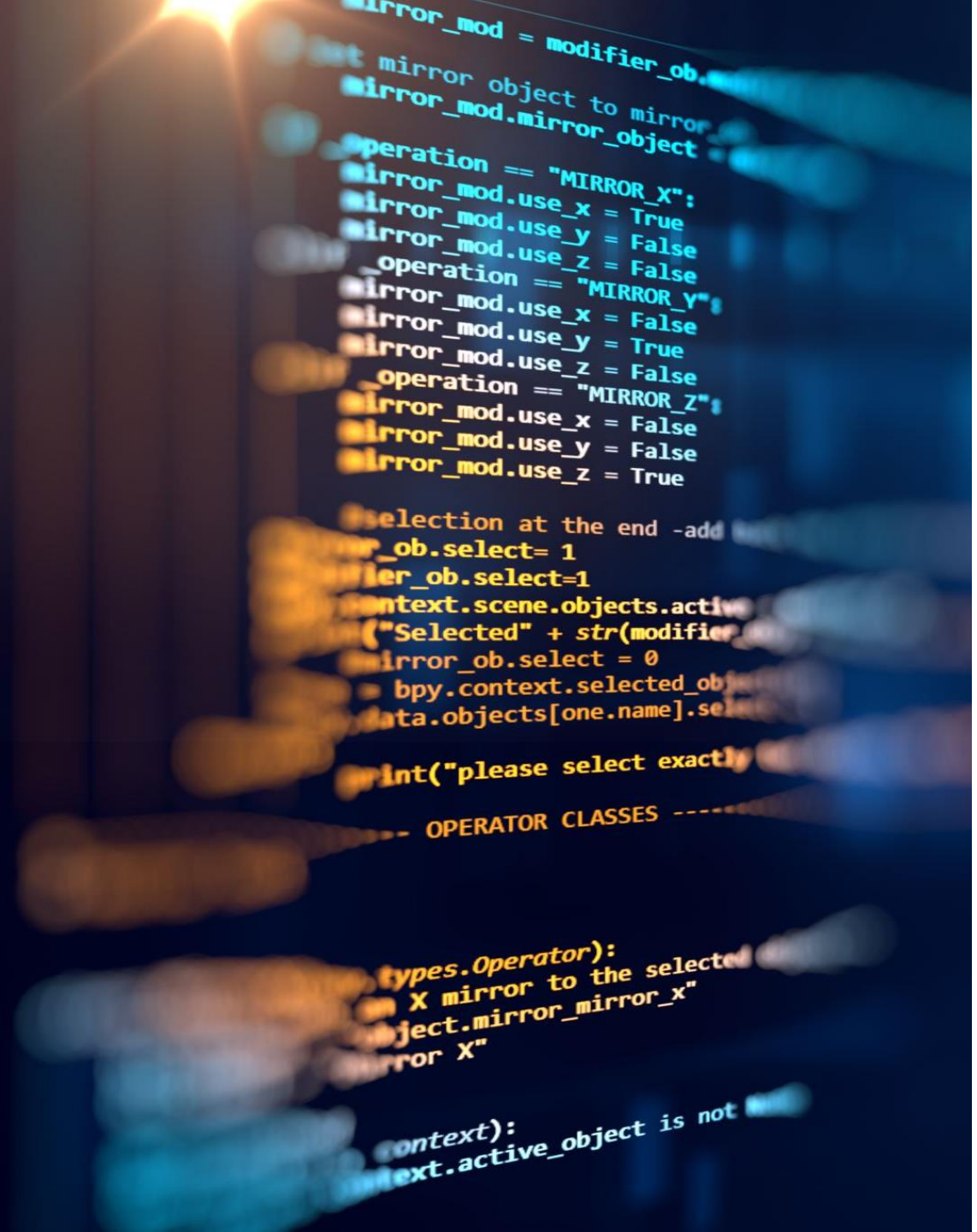
If both are common, the corrected phrase is considered much more likely than the original.

Advantages

- Supports efficient phrase searching.
- Uses word context.
- Improves context-sensitive spell correction.
- Reduces incorrect correction suggestions.

Limitation

- Biword Indexes require additional storage because every adjacent pair of words must also be indexed.



Soundex

- **Soundex** is a **phonetic matching algorithm** used in Information Retrieval to find words that **sound alike**, even if they are spelled differently.
- Instead of comparing the exact spelling of words, Soundex compares **how they are pronounced**.
- This makes it useful when users do not know the correct spelling of a name or word.
- Soundex was originally developed for the **United States Census Bureau** in **1918**.
- Its purpose was to group together records that referred to the same person but used different spellings of the person's name.

Why Do We Need Soundex?

- People often spell names differently while pronouncing them in the same way.
- For example:
  - Smith → Smyth
  - Jon → John
  - Steven → Stephen
- Although the spellings differ, they sound very similar.
- A normal keyword search may fail to retrieve all relevant documents, whereas Soundex can identify these phonetically similar words.

How Does Soundex Work?

- The algorithm converts every word into a **standard phonetic code**.
- Words with the same pronunciation usually receive the same Soundex code.
- The search engine then compares these codes instead of comparing the original spellings.

Applications

- Searching personal names.
- Genealogy databases.
- Government records.
- Census records.
- Customer databases.
- Basic phonetic searching.

Language Dependency

Soundex is **language-specific**.

The original algorithm was designed primarily for **English names**.

It is less effective for languages with different pronunciation rules.

Soundex – Algorithm

Step 1: Keep the First Letter

The first letter of the word is always retained.

Example:

- Robert -> R

Soundex – Algorithm

Step 2: Replace Vowels and Certain Letters with 0

Replace:

- A
- E
- I
- O
- U
- H
- W
- Y

With 0

These letters generally have little effect on pronunciation in the Soundex algorithm.

Soundex – Algorithm

- **Step 3: Replace Remaining Letters with Digits**
- Assign numbers according to pronunciation groups.
- Letters with similar sounds receive the same number.

Letters	Code
B, F, P, V	1
C, G, J, K, Q, S, X, Z	2
D, T	3
L	4
M, N	5
R	6

Example

Word: Robert

Transformation:

- R → Keep
- o → 0
- b → 1
- e → 0
- r → 6
- t → 3

Intermediate code: R01063

Soundex – Algorithm

Remove

Step 4 - Remove Consecutive Duplicate Digits

- If two identical digits appear next to each other, keep only one.
- Example: B1123 -> B123

Remove

Step 5 - Remove All Zeros

- Delete every zero generated from vowels and ignored letters.
- Example: R01063 -> R163

Soundex – Algorithm

- **Step 6: Produce a Four-Character Code**
- If the code contains fewer than four characters, add zeros at the end.
- If it contains more than four characters, keep only the first four.
- Final format: ***Letter Digit Digit Digit***

Example

Herman -> H655

Would **Hermann** produce the same Soundex code?

Since the pronunciation is nearly identical, the answer is **yes**. Both names typically generate the same Soundex code.

Evaluating Soundex

- Soundex is one of the oldest and most widely implemented phonetic matching algorithms.
- It is available in many commercial database systems, including:
  - Oracle
  - Microsoft SQL Server
  - Other relational database systems

Strengths

Soundex works well when searching for:

- Personal names.
- Family names.
- Census records.

Government databases.

It is useful when **high recall** is more important than perfect precision.

For example, law enforcement agencies may prefer retrieving too many possible matches rather than missing an important one.

Limitations

- Although Soundex is useful for phonetic matching, it has several limitations.
  - It was designed mainly for English names.
  - It is less suitable for general Information Retrieval.
  - Different names from other languages may not be encoded accurately.
  - Some unrelated names may receive the same Soundex code.

Research Findings

Zobel and Dart (1996) research showed that **other phonetic matching algorithms often perform better than Soundex in Information Retrieval applications**.

As a result, many modern search systems use more advanced phonetic algorithms.